Recursividade

1. O que é recursividade

Uma função é chamada de **recursiva** quando o corpo dela chama a própria função, de forma **direta** ou **indireta**.

```python
# Recursão direta: funcao1 chama a si mesma
def funcao1():
    ...
    funcao1()
    ...


# Recursão indireta: funcao1 chama funcao2, que chama funcao1 de volta
def funcao1():
    ...
    funcao2()
    ...

def funcao2():
    ...
    funcao1()
    ...
```

Neste material trabalharemos com a recursão **direta**, que é a mais comum.

2. Condição de parada e a pilha de memória

Uma função recursiva resolve um problema reduzindo-o, a cada chamada, a um problema menor do mesmo tipo. Essa redução continua até que seja possível resolver o problema **diretamente**, sem precisar de mais nenhuma chamada. Quando isso acontece, dizemos que o algoritmo atingiu a **condição de parada** (também chamada de **caso base**).

Toda função recursiva precisa ter:

1. **Condição de parada (caso base):** o caso mais simples, resolvido diretamente, sem chamada recursiva.
2. **Chamada recursiva:** a função chama a si mesma, sempre com um problema **menor** que o anterior.

Se a condição de parada nunca for alcançada, a função continua se chamando para sempre. A cada chamada, o Python reserva um espaço na **pilha de memória** (a região da memória que guarda as instâncias de uma função recursiva, cada uma com as suas próprias variáveis

locais). Quando a pilha enche, o programa é encerrado de forma indesejada. É o chamado **estouro de pilha** (*stack overflow*). No Python, isso aparece como o erro `RecursionError`:

```python
def contagem_infinita(n):
    print(n)
    contagem_infinita(n + 1)    # não existe condição de parada!


contagem_infinita(1)
# ... 998, 999, 1000
# RecursionError: maximum recursion depth exceeded
```

O Python limita a profundidade da recursão (por padrão, perto de 1000 chamadas empilhadas). Você pode consultar esse limite com `sys.getrecursionlimit()`.

3. Exemplo 1: fatorial

Na matemática, o fatorial de um número `n`, representado por `n!`, é o produto de todos os inteiros positivos menores ou iguais a `n`.

```
4! = 4 * 3 * 2 * 1 = 24
```

Uma definição recursiva do fatorial é:

```
n! = n * (n - 1)!
```

Dessa forma, para o fatorial de 4, temos:

```
4! = 4 * 3!
3! = 3 * 2!
2! = 2 * 1!
1! = 1 * 0!
0! = 1          <- critério de parada: a forma mais simples do problema
```

Em Python:

```python
```

```python
def fatorial(n):
    if n == 0:                      # condição de parada
        return 1
    else:
        return n * fatorial(n - 1)  # chamada recursiva
```

Passo a passo: `fatorial(4)`

Uma função recursiva trabalha em **duas fases**:

- **Fase 1: descendo.** A função chama a si mesma várias vezes, com valores cada vez menores, até chegar na condição de parada. Enquanto desce, nenhuma conta é concluída: cada chamada **fica esperando** a resposta da seguinte. Os cálculos que ficaram esperando aparecem na última coluna do quadro, com um `?` no lugar da resposta que ainda não existe.
- **Fase 2: voltando.** A condição de parada devolve sua resposta. A partir daí, cada chamada recebe a resposta que estava esperando, faz sua conta e devolve o resultado para quem a chamou. O `?` vai sendo trocado pelo valor real.

Fase 1: descendo

Passo	Chamada	Valor de `n`	Parou? (`n == 0`)	O que acontece	Cálculos esperando resposta
1	`fatorial(4)`	4	Não	Chama `fatorial(3)` e espera	`4 * ?`
2	`fatorial(3)`	3	Não	Chama `fatorial(2)` e espera	`4 * ?` e `3 * ?`
3	`fatorial(2)`	2	Não	Chama `fatorial(1)` e espera	`4 * ?`, `3 * ?` e `2 * ?`
4	`fatorial(1)`	1	Não	Chama `fatorial(0)` e espera	`4 * ?`, `3 * ?`, `2 * ?` e `1 * ?`
5	`fatorial(0)`	0	**Sim**	Devolve `1` sem chamar ninguém. Fim da descida	`4 * ?`, `3 * ?`, `2 * ?` e `1 * ?`

Fase 2: voltando

Passo	Chamada que recebe a resposta	Resposta recebida	Conta feita	Devolve	Cálculos que ainda esperam
6	`fatorial(1)`	`1` (de `fatorial(0)`)	`1 * 1 = 1`	`1`	`4 * ?`, `3 * ?` e `2 * ?`
7	`fatorial(2)`	`1` (de `fatorial(1)`)	`2 * 1 = 2`	`2`	`4 * ?` e `3 * ?`
8	`fatorial(3)`	`2` (de `fatorial(2)`)	`3 * 2 = 6`	`6`	`4 * ?`
9	`fatorial(4)`	`6` (de `fatorial(3)`)	`4 * 6 = 24`	**`24`**	nenhum: resultado final

Por que a condição de parada é `n == 0` e não `n == 1`?

É comum pensar em parar em `n == 1`, porque `1! = 1`. Vamos testar essa versão nos quadros abaixo.

```python
def fatorial(n):
    if n == 1:
        return 1
    else:
        return n * fatorial(n - 1)
```

`fatorial(3)` com parada em `n == 1`: funciona

Passo	Fase	Chamada	O que acontece
1	1 (descendo)	`fatorial(3)`	`n` não é 1. Chama `fatorial(2)` e espera (`3 * ?`)
2	1 (descendo)	`fatorial(2)`	`n` não é 1. Chama `fatorial(1)` e espera (`2 * ?`)
3	1 (descendo)	`fatorial(1)`	`n` é 1: **parou.** Devolve `1`
4	2 (voltando)	`fatorial(2)`	Recebe `1`. Faz `2 * 1 = 2` e devolve `2`
5	2 (voltando)	`fatorial(3)`	Recebe `2`. Faz `3 * 2 = 6` e devolve **`6`**

`fatorial(0)` com parada em `n == 1`: não funciona

Passo	Chamada	Valor de `n`	Parou? (`n == 1`)	O que acontece
1	`fatorial(0)`	0	Não	Chama `fatorial(-1)` e espera
2	`fatorial(-1)`	-1	Não	Chama `fatorial(-2)` e espera
3	`fatorial(-2)`	-2	Não	Chama `fatorial(-3)` e espera
4	`fatorial(-3)`	-3	Não	Chama `fatorial(-4)` e espera
...	...	...	Não	`n` só diminui (0, -1, -2, -3...) e **nunca vale 1**
cerca de 1000	...	...	...	O Python interrompe o programa com `RecursionError`

Repare que esta execução **nunca chega na Fase 2:** sem condição de parada, a descida não termina e nenhuma resposta volta.

`fatorial(0)` com parada em `n == 0` (a versão que usamos)

Passo	Fase	Chamada	O que acontece
1	1 (descendo)	`fatorial(0)`	`n` é 0: **parou.** Devolve `1` na hora

Isso está correto, pois por definição `0! = 1`. E para `fatorial(3)` o resultado continua sendo `6`, com uma chamada a mais (a de `fatorial(0)`, como no quadro do passo a passo acima).

Conclusão: com `n == 0`, a função funciona para o 0 **e** para todos os positivos. Para escolher a condição de parada, pergunte: *"qual é o menor valor de entrada que esta função precisa aceitar?"* e resolva esse caso diretamente. Aqui o menor valor válido é 0, então é nele que a função para.

4. Equação de recorrência

Uma **equação de recorrência** é a definição matemática de uma função recursiva: ela diz quanto vale a função usando o valor da própria função para entradas menores. Para o fatorial:

```
n! = 1,              se n = 0
n! = n * (n - 1)!,   se n > 0
```

- A **primeira linha** é o caso mais simples, resolvido direto: é a condição de parada.

- A **segunda linha** é o caso recursivo: o problema `n!` é reduzido ao problema menor `(n - 1)!`.

Essa forma de escrever é fácil de traduzir para código, porque cada linha da equação vira um ramo do `if`:

```python
def fatorial(n):
    if n == 0:                          # <- "1, se n = 0"           (caso base)
        return 1
    else:                               # <- "n * (n-1)!, se n > 0" (recursão)
        return n * fatorial(n - 1)
```

Veja a equação sendo aplicada para calcular `3!`:

Fase 1: descendo (aplicando a equação até chegar no caso base)

Passo	Expressão	Linha da equação usada	Resultado
1	`3!`	`n > 0`, então `n * (n - 1)!`	`3 * 2!`
2	`2!`	`n > 0`, então `n * (n - 1)!`	`2 * 1!`
3	`1!`	`n > 0`, então `n * (n - 1)!`	`1 * 0!`
4	`0!`	`n = 0`, então vale `1`	`1` (parou)

Fase 2: voltando (trocando cada fatorial pelo valor encontrado)

Passo	Expressão	Substituição	Resultado
5	`1!`	`1 * 0!` vira `1 * 1`	`1`
6	`2!`	`2 * 1!` vira `2 * 1`	`2`
7	`3!`	`3 * 2!` vira `3 * 2`	**`6`**

5. Exemplo 2: somando uma lista

Suponha que desejamos calcular a soma de uma lista de números: `[1, 3, 5, 7, 9]`.

Primeiro, vamos resolver o problema **sem** recursão, com uma variável acumuladora:

```python
```

```python
def soma(lista):
    total = 0                   # (chamamos de "total" para não confundir
    for item in lista:          #  com o nome da função, "soma")
        total = total + item
    return total


lista = [1, 3, 5, 7, 9]
print(soma(lista))              # 25
```

Pensando de forma recursiva

Imagine que **não** temos disponíveis os laços `while` e `for`. Como somar a lista?

Do ponto de vista matemático, a adição é uma função definida por um **par** de números. Para redefinir o problema da adição de uma lista, podemos reescrever a lista como uma expressão de somas entre parênteses:

```
total = (1+(3+(5+(7+9))))
```

O par de parênteses mais interno, `(7+9)`, é um problema que sabemos resolver sem laço nenhum. Daí, basta ir simplificando:

Etapa	Expressão	O que foi feito
1	`(1+(3+(5+(7+9))))`	Expressão original
2	`(1+(3+(5+16)))`	Resolveu o par mais interno: `7 + 9 = 16`
3	`(1+(3+21))`	Resolveu o par mais interno: `5 + 16 = 21`
4	`(1+24)`	Resolveu o par mais interno: `3 + 21 = 24`
5	`25`	Resolveu o último par: `1 + 24 = 25`

Para transformar essa ideia em uma função recursiva, dizemos que **a soma da lista é o primeiro número somado com a soma do restante da lista**. Em Python:

- o primeiro número é `lista[0]`;
- o restante da lista (sem o primeiro item) é `lista[1:]`.

Falta decidir quando parar. Na expressão `(1+(3+(5+(7+9))))`, paramos no par `(7+9)`. Para o computador, o jeito mais simples e seguro é parar quando a lista **acaba**: a soma de uma lista vazia é `0`. Com isso, o último número também vira "número + soma do restante (vazio)", e a expressão fica assim:

```
total = (1+(3+(5+(7+(9+0)))))
```

Em Python:

```python
def soma(lista):
    if len(lista) == 0:                    # condição de parada: lista vazia
        return 0
    else:
        return lista[0] + soma(lista[1:])   # primeiro + soma do restante
```

Passo a passo: `soma([1, 3, 5, 7, 9])`

Fase 1: descendo

Passo	Chamada	Lista vazia?	O que acontece	Cálculos esperando resposta
1	`soma([1, 3, 5, 7, 9])`	Não	Separa o `1`, chama `soma([3, 5, 7, 9])` e espera	`1 + ?`
2	`soma([3, 5, 7, 9])`	Não	Separa o `3`, chama `soma([5, 7, 9])` e espera	`1 + ?` e `3 + ?`
3	`soma([5, 7, 9])`	Não	Separa o `5`, chama `soma([7, 9])` e espera	`1 + ?`, `3 + ?` e `5 + ?`
4	`soma([7, 9])`	Não	Separa o `7`, chama `soma([9])` e espera	`1 + ?`, `3 + ?`, `5 + ?` e `7 + ?`
5	`soma([9])`	Não	Separa o `9`, chama `soma([])` e espera	`1 + ?`, `3 + ?`, `5 + ?`, `7 + ?` e `9 + ?`
6	`soma([])`	**Sim**	Devolve `0` sem chamar ninguém. Fim da descida	`1 + ?`, `3 + ?`, `5 + ?`, `7 + ?` e `9 + ?`

Fase 2: voltando

Passo	Chamada que recebe a resposta	Resposta recebida	Conta feita	Devolve	Cálculos que ainda esperam
7	`soma([9])`	`0` (de `soma([])`)	`9 + 0 = 9`	`9`	`1 + ?`, `3 + ?`, `5 + ?` e `7 + ?`
8	`soma([7, 9])`	`9` (de `soma([9])`)	`7 + 9 = 16`	`16`	`1 + ?`, `3 + ?` e `5 + ?`
9	`soma([5, 7, 9])`	`16` (de `soma([7, 9])`)	`5 + 16 = 21`	`21`	`1 + ?` e `3 + ?`
10	`soma([3, 5, 7, 9])`	`21` (de `soma([5, 7, 9])`)	`3 + 21 = 24`	`24`	`1 + ?`
11	`soma([1, 3, 5, 7, 9])`	`24` (de `soma([3, 5, 7, 9])`)	`1 + 24 = 25`	**`25`**	nenhum: resultado final

Por que parar na lista vazia e não na lista com 1 item?

Também é comum pensar em parar quando sobra **1 item** (`len(lista) == 1`), devolvendo esse item. Vamos testar essa versão nos quadros abaixo.

```python
def soma(lista):
    if len(lista) == 1:
        return lista[0]
    else:
        return lista[0] + soma(lista[1:])
```

`soma([3, 5])` com parada em `len(lista) == 1`: funciona

Passo	Fase	Chamada	O que acontece
1	1 (descendo)	`soma([3, 5])`	Tem 2 itens, não é 1. Separa o `3`, chama `soma([5])` e espera (`3 + ?`)
2	1 (descendo)	`soma([5])`	Tem 1 item: **parou**. Devolve `5`
3	2 (voltando)	`soma([3, 5])`	Recebe `5`. Faz `3 + 5 = 8` e devolve **`8`**

`soma([])` com parada em `len(lista) == 1`: não funciona

Passo	Chamada	`len(lista)`	Parou? (`len == 1`)	O que acontece
1	`soma([])`	0	Não	Cai no `else` e tenta ler `lista[0]`. Numa lista vazia **não existe** primeiro item: o Python interrompe com `IndexError: list index out of range`

`soma([])` com parada em `len(lista) == 0` (a versão que usamos)

Passo	Fase	Chamada	O que acontece
1	1 (descendo)	`soma([])`	`len(lista)` é 0: **parou**. Devolve `0` na hora

Para as outras listas o resultado é o mesmo da versão com 1 item: no quadro da Fase 2 do passo a passo, o passo 7 apenas acrescenta um `+ 0`, que não muda a soma.

Conclusão: a lista vazia é o menor caso possível. Ao resolvê-lo diretamente, a função funciona para **qualquer** lista, vazia ou não. (É a mesma pergunta do fatorial: *qual é o menor valor de entrada que a função precisa aceitar?*)

6. Divisão e conquista

Divisão e conquista é uma estratégia para resolver problemas grandes: em vez de resolver tudo de uma vez, o problema é **quebrado em partes menores, do mesmo tipo do original**. Cada parte é resolvida e, no fim, as respostas das partes são **juntadas** para formar a resposta do problema inteiro.

Um exemplo do dia a dia: contar as folhas de uma pilha de 1.000 folhas. Contar uma a uma é lento e fácil de errar. Pela estratégia de divisão e conquista, você separa a pilha em duas de 500 e entrega cada uma a uma pessoa. Cada pessoa também separa a sua pilha em duas e repassa, e assim por diante, até que cada pilha seja tão pequena que a contagem seja imediata (uma folha). Aí as respostas voltam, e cada pessoa soma o que recebeu das duas pilhas que repassou.

As três etapas

Etapa	O que faz	Onde aparece na função recursiva
1. **Dividir**	Quebra o problema em partes menores, do mesmo tipo do original	As chamadas recursivas, que acontecem na **Fase 1 (descendo)**
2. **Conquistar**	Resolve as partes. Se a parte ainda é grande, divide de novo. Se é pequena o bastante para ter resposta direta, resolve na hora	A **condição de parada**
3. **Combinar**	Junta as respostas das partes para formar a resposta do problema maior	A conta feita na **Fase 2 (voltando)**: multiplicar, somar, comparar...

Os exemplos que já vimos usam essa estratégia

O fatorial e a soma de lista já são divisão e conquista, na forma mais simples: o problema é dividido em **um pedaço pequeno** (o número `n`, ou o primeiro item da lista) e **um problema menor** (o `(n - 1)!`, ou o restante da lista).

Problema	Dividir	Conquistar (condição de parada)	Combinar
`fatorial(n)`	Separa `n` e o problema `fatorial(n - 1)`	`n == 0` devolve `1`	`n * resposta`
`soma(lista)`	Separa o primeiro item e o problema `soma(restante)`	Lista vazia devolve `0`	`primeiro + resposta`

Dividindo ao meio

Outra forma de dividir é **cortar o problema em duas metades**, de tamanho parecido. Veja a soma de uma lista feita assim:

```python
```

```python
def soma_dividida(lista: list[int]) -> int:
    """Soma os itens de uma lista dividindo-a ao meio, de forma recursiva.

    Args:
        lista: lista de números inteiros.

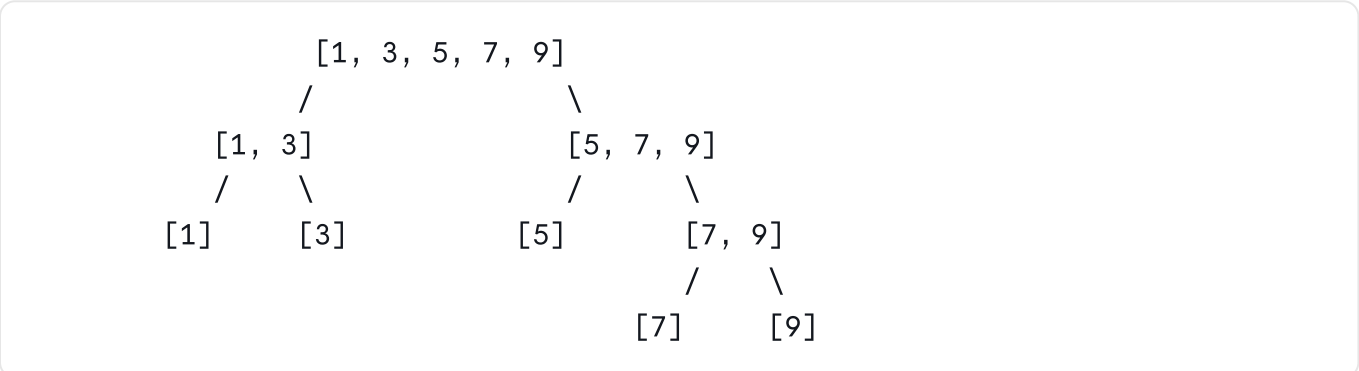
    Returns:
        O somatório dos números da lista (0 se a lista estiver vazia).
    """
    if len(lista) == 0:                # parada 1: lista vazia
        return 0
    if len(lista) == 1:                # parada 2: um item, a soma é ele mesmo
        return lista[0]

    meio = len(lista) // 2             # posição do corte
    esquerda = lista[:meio]            # primeira metade
    direita = lista[meio:]             # segunda metade
    return soma_dividida(esquerda) + soma_dividida(direita)   # combina as duas
```

Detalhes do código:

- `len(lista) // 2` é a divisão inteira (sem casas decimais). Para uma lista de 5 itens, `5 // 2` vale `2`.
- `lista[:meio]` pega os itens **antes** da posição `meio` e `lista[meio:]` pega os itens **a partir** dela. Com `lista = [1, 3, 5, 7, 9]` e `meio = 2`, `esquerda` é `[1, 3]` e `direita` é `[5, 7, 9]`.
- Existem **duas chamadas recursivas**, uma para cada metade. A etapa de combinar é a soma das duas respostas.

A divisão de `soma_dividida([1, 3, 5, 7, 9])` forma uma árvore:



Fase 1: dividindo (de cima para baixo)

Nível	Partes neste nível	O que acontece
0	`[1, 3, 5, 7, 9]`	Tem 5 itens. `meio = 2`: divide em `[1, 3]` e `[5, 7, 9]`
1	`[1, 3]` e `[5, 7, 9]`	`[1, 3]` divide em `[1]` e `[3]`. `[5, 7, 9]` tem 3 itens, `meio = 1`: divide em `[5]` e `[7, 9]`
2	`[1]`, `[3]`, `[5]` e `[7, 9]`	`[1]`, `[3]` e `[5]` têm 1 item: **pararam** (a soma é o próprio item). `[7, 9]` divide em `[7]` e `[9]`
3	`[7]` e `[9]`	Têm 1 item: **pararam**

Fase 2: combinando (de baixo para cima)

Passo	Parte	Respostas das duas metades	Conta feita	Resposta
1	`[7, 9]`	`7` e `9`	`7 + 9`	`16`
2	`[1, 3]`	`1` e `3`	`1 + 3`	`4`
3	`[5, 7, 9]`	`5` e `16`	`5 + 16`	`21`
4	`[1, 3, 5, 7, 9]`	`4` e `21`	`4 + 21`	**`25`**

> **Observação:** os quadros mostram a árvore por níveis para facilitar o entendimento. Na prática, o Python resolve **um lado de cada vez**: primeiro toda a metade esquerda, depois a direita. Por isso, aqui a divisão e a combinação se alternam, diferente do fatorial e da soma "um a um", em que primeiro tudo desce e depois tudo volta. O resultado final é o mesmo.

Por que precisa da parada em `len(lista) == 1`? Cada parte precisa ser **realmente menor** que o problema que a gerou. Veja o que acontece se a função parasse apenas na lista vazia, com `soma_dividida([5])`:

Passo	Chamada	Tamanho	`meio`	`esquerda` e `direita`	O que acontece
1	`soma_dividida([5])`	1	`1 // 2 = 0`	`[]` e `[5]`	A `direita` é igual ao problema original: chama `soma_dividida([5])` de novo
2	`soma_dividida([5])`	1	`0`	`[]` e `[5]`	Mesma situação: chama `soma_dividida([5])` de novo
...	...	...	...	...	O problema **nunca diminui**
cerca de 1000	...	...	...	...	`RecursionError`

Outro exemplo: o maior valor de uma lista

A etapa de combinar não precisa ser uma soma. Para achar o **maior valor** de uma lista, cada metade devolve o seu maior valor e a combinação é **escolher o maior dos dois**:

python

```python
def maior_valor(lista: list[int]) -> int:
    """Encontra o maior valor de uma lista dividindo-a ao meio, de forma recurs

    Args:
        lista: lista de números inteiros com pelo menos um item.

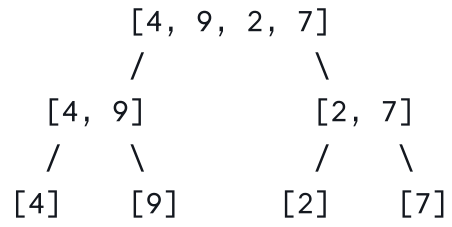
    Returns:
        O maior número da lista.
    """
    if len(lista) == 1:                          # parada: um item, ele é o maio
        return lista[0]

    meio = len(lista) // 2
    maior_esquerda = maior_valor(lista[:meio])  # maior da primeira metade
    maior_direita = maior_valor(lista[meio:])   # maior da segunda metade

    if maior_esquerda > maior_direita:           # combina: escolhe o maior dos
        return maior_esquerda
    else:
        return maior_direita
```

Aqui a lista vazia não é um caso válido (uma lista sem itens não tem maior valor), então a menor entrada aceita é a lista com 1 item.

Árvore de `maior_valor([4, 9, 2, 7])`:



Fase 1: dividindo (de cima para baixo)

Nível	Partes neste nível	O que acontece
0	`[4, 9, 2, 7]`	Tem 4 itens. `meio = 2`: divide em `[4, 9]` e `[2, 7]`
1	`[4, 9]` e `[2, 7]`	Cada uma tem 2 itens, `meio = 1`: dividem em `[4]`, `[9]`, `[2]` e `[7]`
2	`[4]`, `[9]`, `[2]` e `[7]`	Têm 1 item: **pararam** (o maior é o próprio item)

Fase 2: combinando (de baixo para cima)

Passo	Parte	Respostas das duas metades	Conta feita	Resposta
1	`[4, 9]`	`4` e `9`	maior entre `4` e `9`	`9`
2	`[2, 7]`	`2` e `7`	maior entre `2` e `7`	`7`
3	`[4, 9, 2, 7]`	`9` e `7`	maior entre `9` e `7`	**`9`**

Quando dividir ao meio compensa

Dividir ao meio faz a pilha de chamadas crescer **muito menos**. Veja o maior número de chamadas abertas ao mesmo tempo para somar uma lista:

Itens na lista	`soma` (um a um)	`soma_dividida` (ao meio)
8	9 chamadas	4 chamadas
100	101 chamadas	8 chamadas
1000	1001 chamadas: `RecursionError`	11 chamadas

Cada vez que a lista **dobra** de tamanho, a versão ao meio precisa de apenas **1 chamada a mais** na pilha, enquanto a versão um a um dobra o número de chamadas. Algoritmos muito usados na prática, como a busca binária e a ordenação por mesclagem (*merge sort*), usam essa mesma estratégia.

Resumo dos exemplos

Problema	Dividir	Conquistar (condição de parada)	Combinar
`fatorial(n)`	`n` e `fatorial(n - 1)`	`n == 0` devolve `1`	`n * resposta`
`soma(lista)`	Primeiro item e restante	Lista vazia devolve `0`	`primeiro + resposta`
`soma_dividida(lista)`	Duas metades	Lista com 1 item devolve o item (vazia devolve `0`)	`resposta da esquerda + resposta da direita`
`maior_valor(lista)`	Duas metades	Lista com 1 item devolve o item	O maior entre as duas respostas

7. Vantagens e desvantagens da recursão

Vantagens

- **Clareza e intuitividade:** a recursão traduz diretamente definições matemáticas, o que deixa o código mais fácil de entender para quem conhece a base teórica.
- **Menos linhas de código:** soluções recursivas costumam ser mais concisas que as versões iterativas.
- **Soluções elegantes:** alguns problemas (como percorrer estruturas aninhadas) têm soluções muito mais simples quando abordados de forma recursiva.

Desvantagens

- **Consumo de memória:** cada chamada guarda suas variáveis na pilha, então soluções recursivas podem consumir mais memória.
- **Desempenho:** por causa da alocação de memória a cada chamada, as soluções recursivas podem ser mais lentas que as iterativas.
- **Estouro de pilha:** chamadas em excesso podem esgotar a pilha (`RecursionError`).
- **Dificuldade de depuração:** rastrear um problema é mais difícil, pois muitas chamadas da mesma função se acumulam.

8. Exemplos de mercado

A recursão aparece sempre que os dados têm uma estrutura "dentro de outra", ou seja, **aninhada**:

- **Pastas e arquivos:** listar tudo que existe dentro de uma pasta exige entrar em cada subpasta, e depois nas subpastas das subpastas.
- **Menus de categorias de lojas virtuais:** "Roupas > Camisetas > Manga Curta" é uma árvore de categorias, exatamente como a que construiremos no projeto de hoje.
- **Organogramas e árvores genealógicas:** cada pessoa tem uma lista de subordinados (ou filhos), que por sua vez também têm.
- **Respostas de APIs e arquivos JSON aninhados:** muitos JSONs trazem objetos dentro de objetos, com profundidade que não conhecemos de antemão. Uma função recursiva percorre qualquer um deles.
- **Comentários em respostas de comentários:** redes sociais e fóruns exibem conversas em que cada comentário pode ter respostas.

---

9. Exercícios principais

Os exercícios abaixo estão em ordem **crescente de dificuldade**. Todos devem ser resolvidos com **funções recursivas**.

Exercício 1: Somatório de 1 a N

Escreva uma função recursiva em Python que recebe como entrada um número inteiro positivo `N` e retorna o somatório de todos os números inteiros no intervalo de 1 a `N`.

Por exemplo, para `N = 5`: `5 + 4 + 3 + 2 + 1 = 15`.

Exercício 2: Potência

Escreva uma função recursiva em Python que receba dois números inteiros `A` e `B` e calcule o resultado de `A^B`. Considere que o resultado de `A^B` é igual a `A * A^(B-1)`.

Por exemplo: o resultado de `4^5` pode ser calculado como:

```
4^5 = 4 * 4^4
4^4 = 4 * 4^3
4^3 = 4 * 4^2
4^2 = 4 * 4^1
4^1 = 4 * 4^0
4^0 = 1
```

Exercício 3: Busca em uma lista

Escreva uma função recursiva para realizar uma busca em uma lista de números. A função deve receber como parâmetros a lista e um item para buscar na lista. Deve retornar `True` caso o item seja localizado, ou `False`, caso não seja localizado.

Exercício 4: Contando ocorrências

Escreva uma função recursiva que recebe como parâmetros uma lista de números inteiros e um número inteiro `N`. A função deve retornar a quantidade de vezes que `N` aparece na lista.

Exercício 5: Máximo divisor comum (MDC)

O **máximo divisor comum (MDC)** de dois números inteiros `x` e `y` é o maior inteiro que **divide** `x` e `y` ao mesmo tempo, e pode ser calculado de acordo com a definição abaixo. Escreva uma função recursiva em Python que calcule o MDC de dois números inteiros positivos.

```
mdc(x, y) = y,              se x >= y e x % y == 0
            mdc(y, x),      se x < y
            mdc(y, x % y), caso contrário
```

Por exemplo, o MDC de 12 e 18 é 6, pois 6 é o maior número que divide 12 e 18 ao mesmo tempo.

---

10. Exercícios complementares

Os exercícios abaixo são **complementares**: reforçam o conceito com listas e com estruturas aninhadas (JSON). O último é o mais desafiador da aula.

Exercício complementar 1: Multiplicação por somas sucessivas

Escreva uma função recursiva que recebe como parâmetros dois números inteiros e retorna o resultado da multiplicação entre esses dois números. Realize o cálculo utilizando **somas sucessivas.**

Por exemplo: `4 * 5 = 5 + 5 + 5 + 5 = 20`.

Exercício complementar 2: Soma dos elementos de uma lista

Defina uma função recursiva que recebe como parâmetro uma lista de números inteiros e retorna o somatório dos números contidos na lista.

Exercício complementar 3: Soma dos números pares de uma lista

Defina uma função recursiva que recebe como parâmetro uma lista de números inteiros e retorna o somatório dos números **pares** contidos na lista.

Exercício complementar 4: Árvore genealógica em JSON

Considere um arquivo JSON que representa a árvore genealógica de uma família. Cada pessoa é representada como um dicionário com a chave `"nome"` para o nome e a chave `"filhos"` (se houver) para uma lista de filhos. Cada filho também é um dicionário.

Criar uma função recursiva que percorre um arquivo JSON nesse formato e exibe os nomes de todas as pessoas da família. Considere diferentes arquivos, com diferentes níveis hierárquicos (`familia1.json` a `familia5.json`, fornecidos junto com a aula).

Exemplo de arquivo JSON (`familia2.json`):

```json
```

```
{
  "nome": "Antônio Lima",
  "filhos": [
    {
      "nome": "Paula Lima",
      "filhos": [
        {
          "nome": "Bruno Lima",
          "filhos": [
            {
              "nome": "Davi Lima"
            }
          ]
        },
        {
          "nome": "Larissa Lima"
        }
      ]
    },
    {
      "nome": "Cláudio Lima",
      "filhos": [
        {
          "nome": "Sofia Lima"
        }
      ]
    }
  ]
}
```

Dica: o JSON lido com `json.load()` (Aula 10) vira um **dicionário**, e cada item da lista `"filhos"` é um dicionário no mesmo formato do original. É um problema "dentro de outro igual", o cenário ideal para recursão.

---

11. Evolução do projeto de fundo: E-commerce simples

Assim como nas Aulas 08, 09, 10 e 11, a Aula 12 apenas **adiciona** funcionalidades novas ao projeto, sem alterar nenhuma função que já existia.

Lojas virtuais organizam seus produtos em **categorias** que têm **subcategorias**, que por sua vez têm outras subcategorias. Essa estrutura é uma **árvore**, representada como nas árvores genealógicas em JSON: cada categoria é um dicionário com a chave `"nome"` e, se tiver subcategorias, a chave `"filhos"` com uma lista de dicionários no mesmo formato.

Exemplo de árvore de categorias:

```
Loja
├── Roupas
|   ├── Camisetas
|   |   ├── Manga Curta
|   |   └── Manga Longa
|   └── Bonés
├── Calçados
|   ├── Tênis
|   |   ├── Corrida
|   |   └── Casual
|   └── Sandálias
└── Acessórios
    └── Mochilas
```

Enunciado

1. Crie a função `listar_categorias`, que recebe uma categoria (o dicionário da árvore) e um parâmetro `nivel` (valor padrão `0`) e exibe os nomes de todas as categorias, com um recuo proporcional ao nível em que cada uma está.
2. Crie a função `contar_categorias`, que recebe uma categoria e devolve o total de categorias da árvore, contando a própria categoria recebida e todas as suas descendentes.
3. Crie a função `buscar_caminho_categoria`, que recebe uma categoria e o nome de uma categoria procurada, e devolve a lista com o caminho da raiz até a categoria procurada (por exemplo, `["Loja", "Calçados", "Tênis"]`), ou `None` caso a categoria não exista na árvore.